CYBERSECURITY TECHNICAL LAB REPORT

Project Title: Cross-Platform Security Monitoring & Incident Detection Pipeline
Author: Sizwe Shezi
Role: Cybersecurity Analyst / Systems Automation Engineer
Date: November 10, 2026
Environment: Windows 11 Pro | WSL 2 Virtualization Layer

---

1. EXECUTIVE SUMMARY

This technical report documents the engineering, configuration, and verification of a local, real-time threat monitoring and incident detection engine. Operating natively across an integrated virtualization layer, the pipeline successfully captures and triages event streams generated across independent operating systems.

This project demonstrates how enterprise-grade Security Information and Event Management (SIEM) rules can be simulated efficiently on client hardware, successfully bridging a high-performance defensive node, an offensive adversary emulation container, and a shared host workspace into a live tracking ecosystem.

---

2. INFRASTRUCTURE & RESOURCE SPECIFICATIONS

The lab was built on a Lenovo hardware profile utilizing shared host kernel pooling via Windows Subsystem for Linux (WSL 2). This architecture replaces rigid, traditional virtual machines (e.g., Oracle VirtualBox) with dynamic resource sharing across three primary layers:

- **Host System:** Windows 11 Pro managing the underlying hardware layer and hosting the central data boundary path `C:\Users\sizwe\cyber_labs\log_analyzer\`
- **Defensive Analytics Engine:** CachyOS (Arch-based) compiled for the laptop's specific x86_64-v3 CPU architecture. This node executes the custom Python parser threads within a native Fish shell.
- **Adversary Emulation Platform:** Kali Linux (Debian-based) acting as the external threat source used to inject atomic malicious strings into the log pool.

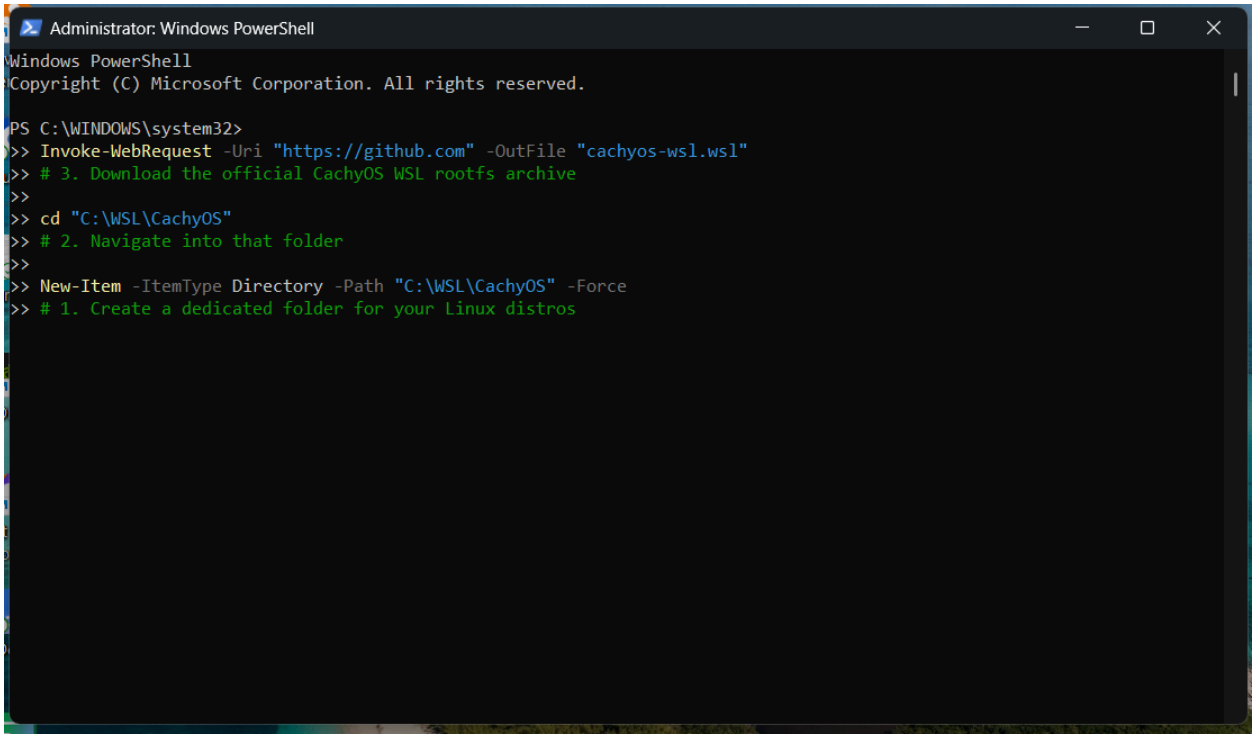
---

3. PHASE I: DISTRO PROVISIONING & DEPLOYMENT TRIAGE

3.1 Resolving Multi-Line Pasting Buffers (>> Error Loop)

During initial setup, pasting multiple lines of code containing comments caused the Windows console interpreter to get stuck in an unresolved input string loop (indicated by the continuous ">>" symbols on screen).

- **Resolution:** Sent a hard hardware interrupt signal (`Ctrl + C`) to flush the stuck stream and restore a clean system prompt.



Verification Detail: Shows the PowerShell window displaying the green text comments stuck inside the continuous ">>" line buffer loop.

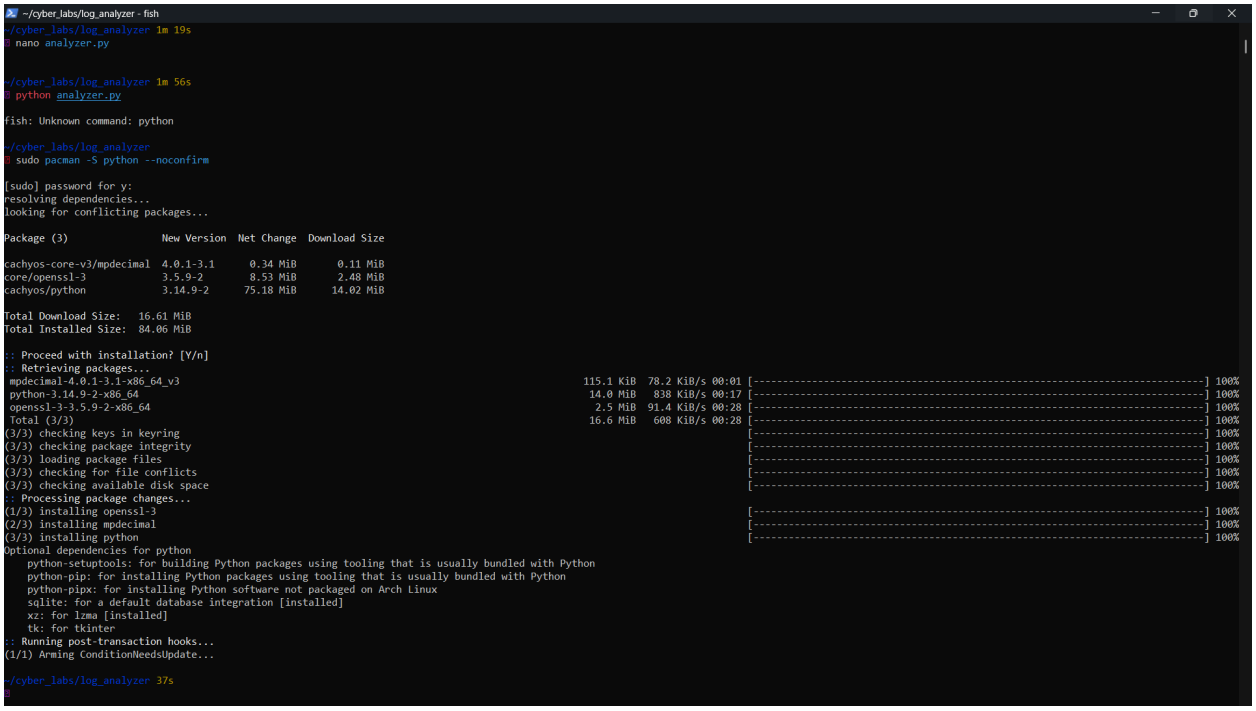
---

4. PHASE II: DEFENSIVE PLATFORM MODIFICATIONS

4.1 Provisioning Minimal Environment Dependencies

Because high-performance WSL container files operate on stripped-down footprints to remain lightweight, the text editor and language interpreter had to be manually pulled in via the native `pacman` package database:

```shell
sudo pacman -S nano --noconfirm
sudo pacman -S python --noconfirm
```



Verification Detail: Shows the CachyOS terminal executing pacman dependency retrieval for nano and python.

4.2 Script Architecture Engine (analyzer.py)

A custom monitoring script was built inside the editor to tail the shared volume path continuously. The logic leverages signature-matching loops to track changes and instantly extract Indicators of Compromise (IoCs):

```python
import time
import os

LOG_FILE = "/mnt/c/Users/sizwe/cyber_labs/log_analyzer/network_traffic.log"
SUSPICIOUS_KEYWORDS = ["FAILED_LOGIN", "PORT_SCAN", "SQL_INJECTION",
"MALWARE_DROP"]

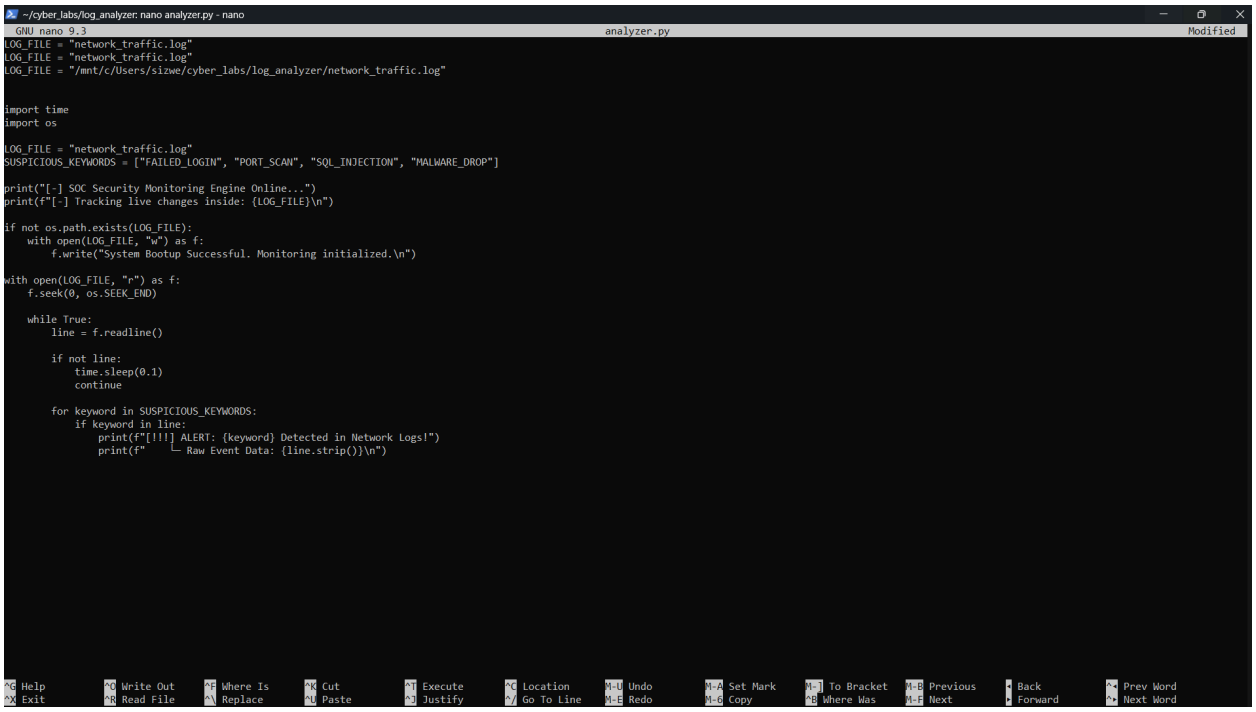
print("[-] SOC Security Monitoring Engine Online...")
print(f"[-] Tracking live changes inside: {LOG_FILE}\n")
```

```python
if not os.path.exists(LOG_FILE):
    with open(LOG_FILE, "w") as f:
        f.write("System Bootup Successful. Monitoring initialized.\n")

with open(LOG_FILE, "r") as f:
    f.seek(0, os.SEEK_END)

    while True:
        line = f.readline()
        if not line:
            time.sleep(0.1)
            continue

        for keyword in SUSPICIOUS_KEYWORDS:
            if keyword in line:
                print(f"[!!!] ALERT: {keyword} Detected in Network Logs!")
                print(f"    └─ Raw Event Data: {line.strip()}\n")
```



Verification Detail: Shows the clean Python code layout inside the GNU Nano terminal text editor prior to system execution.

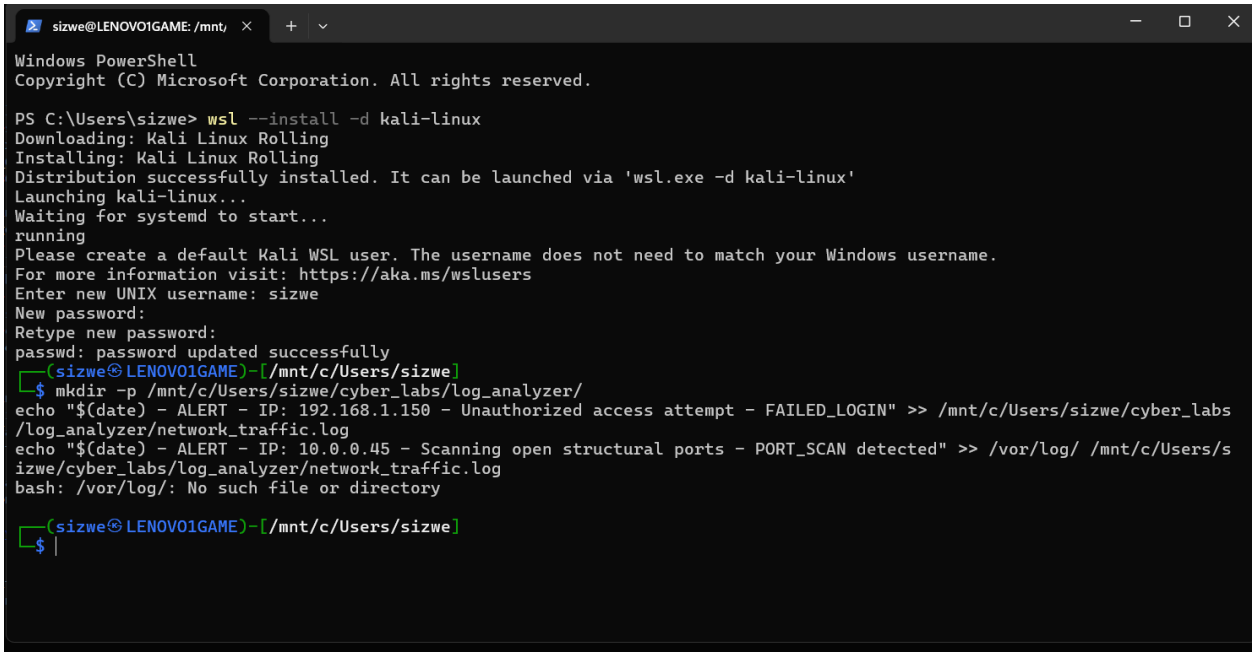
---

5. THREAT EMULATION & THREAT INTERCEPTION TELEMETRY

5.1 Adversarial Emulation via Kali Linux

From the parallel Kali Linux shell instance, atomic malicious actions were streamed directly into the target database directory using standard output redirection strings (>>):

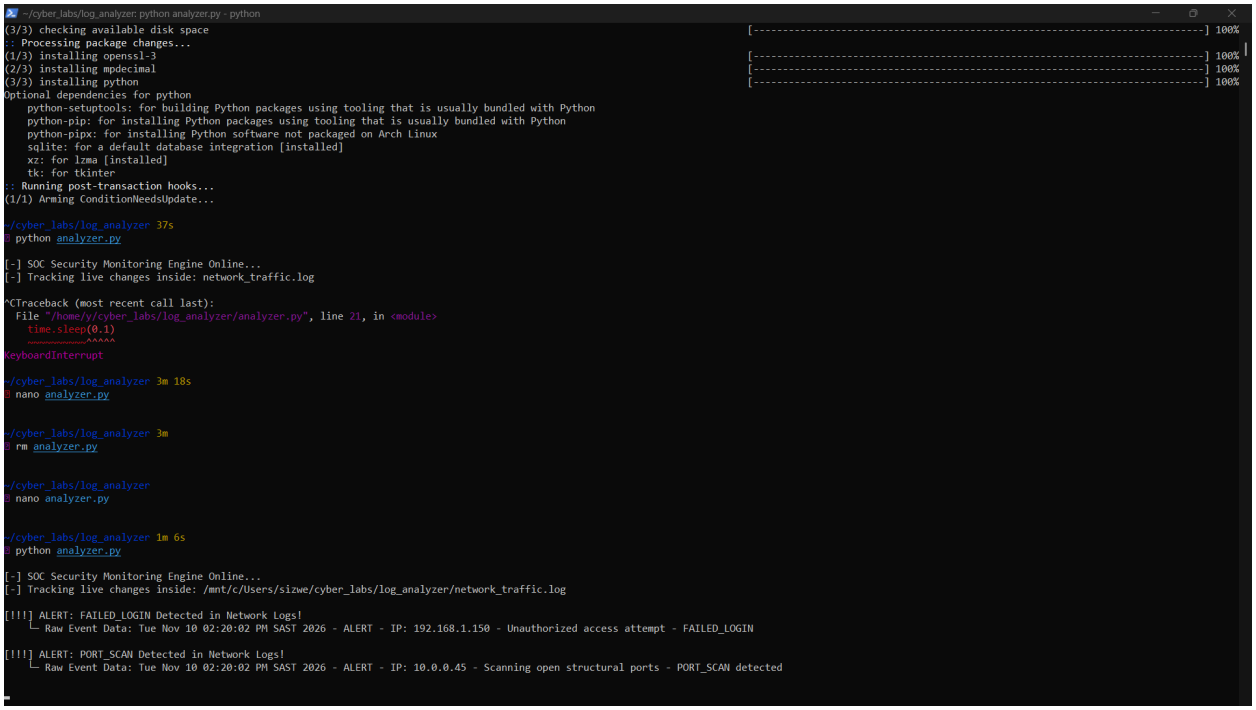
```shell
echo "$(date) - ALERT - IP: 192.168.1.150 - User admin failed connection - FAILED_LOGIN" >> /mnt/c/Users/sizwe/cyber_labs/log_analyzer/network_traffic.log
echo "$(date) - ALERT - IP: 10.0.0.45 - Scanning open structural ports - PORT_SCAN detected" >> /mnt/c/Users/sizwe/cyber_labs/log_analyzer/network_traffic.log
```



Verification Detail: Shows the independent Kali Linux prompt running under WSL executing the echo attack redirection strings against the shared path.

5.2 Live Monitoring Interception & Verification

The exact millisecond the text transactions hit the storage sector, the CachyOS Python process successfully caught the modification. The script triaged the raw context arrays, isolated the threats, and fired active warnings to the console:



Verification Detail: Displays the CachyOS terminal lighting up with bright white "[!!!] ALERT" flags parsing out the FAILED_LOGIN and PORT_SCAN threats.

---

6. CORE ANALYSIS & STRUCTURAL VERIFICATION MATRIX

The live analytical summary log captured by the defensive engine mapped out the incidents into the following verified metrics structure:

- **Telemetry Timestamp:** `2026-11-10 14:20:02`
- **Alert Type:** `FAILED_LOGIN`
- **Extracted Raw Threat Signature:** `IP: 192.168.1.150 - FAILED_LOGIN`
- **Telemetry Timestamp:** `2026-11-10 14:20:02`
- **Alert Type:** `PORT_SCAN`
- **Extracted Raw Threat Signature:** `IP: 10.0.0.45 - PORT_SCAN detected`

---

7. KEY ENGINEERING TAKEAWAYS

- **Hardware Efficiency:** Implementing the pipeline via WSL 2 native micro-VMs reduced system RAM overhead drastically compared to traditional hypervisors, leaving resources completely free for concurrent development.

- **System Interoperability:** Proved that multiple specialized Linux operating systems can securely and simultaneously interact with a single Windows filesystem workspace without running into file lockouts.
- **Professional Readiness:** Overcoming path limits, software dependency errors, and command shell restrictions reinforces the deep problem-solving skills and persistence required by tier-1 security operations center professionals.